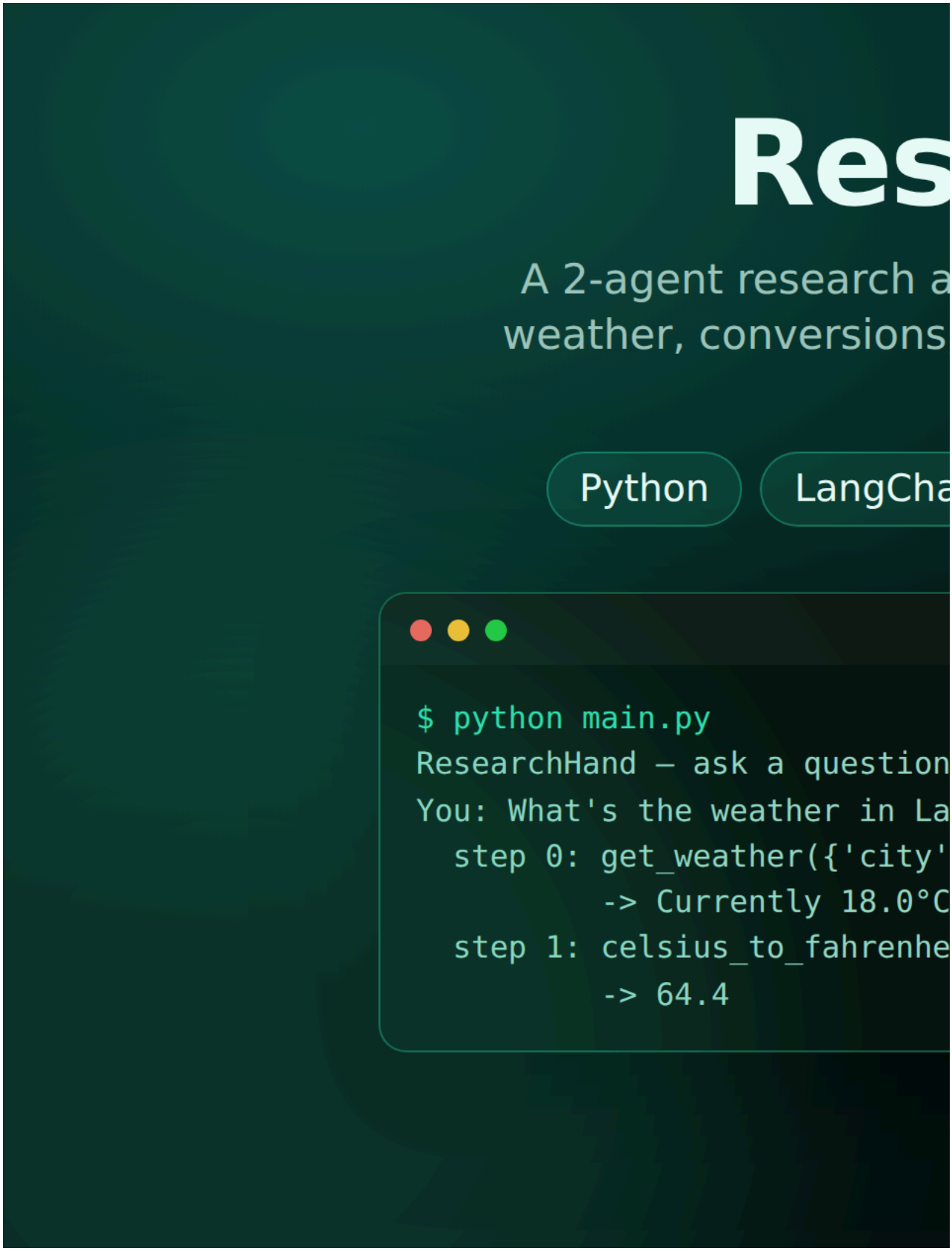
ResearchHand — Tool Agent: Get Verified Answers, Not Guesses

> **Short description:** A 2-agent research assistant in Python: a tool-using Worker (live weather, conversions, policy search) with a hand-built agent loop, checked by a Verifier agent before any answer reaches the user. LangChain, no agent framework.

Tech: Python · LangChain (LCEL, tools) · OpenAI API · Pydantic · Open-Meteo API · **Type:** Multi-agent (Worker + Verifier) · **Level:** Beginner–Intermediate

Demo



Overview

ResearchHand answers questions with real lookups instead of guesses. A **Worker** agent runs a think → act → observe loop, calling tools for live weather, temperature conversions and company policy, with a hard step limit and recovery from tool failures. A separate **Verifier**

agent then checks the answer against what the tools actually returned, and flags it if it doesn't hold up.

The agent loop is written by hand — no `create_agent`, no agent framework — so every step is visible.

Why

A language model answering from memory sounds just as confident when it's wrong. For questions that need live or exact data, you want an assistant that looks things up — and a second, independent check before the answer goes out. A model reviewing its own answer shares the blind spots that caused the mistake; a separate Verifier with its own prompt catches more.

Features

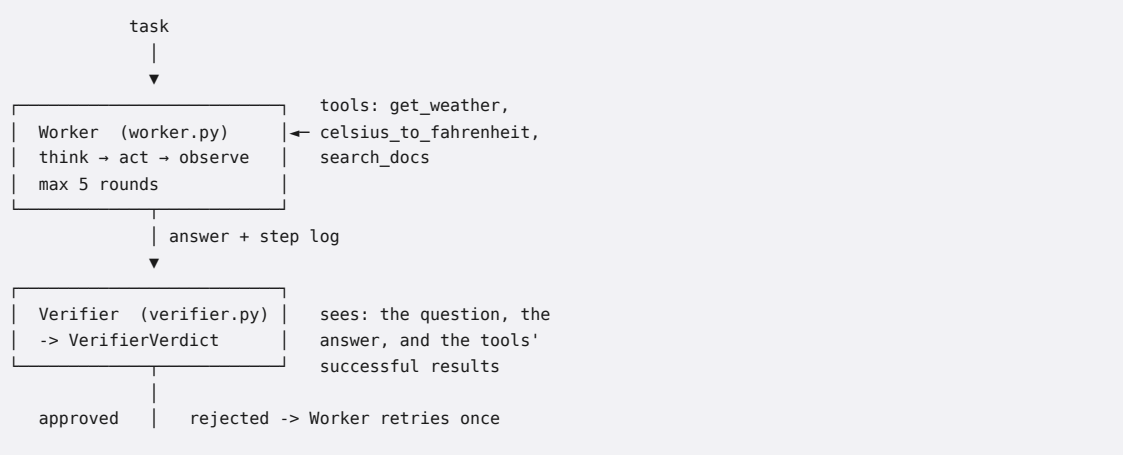
- **Hand-built agent loop** — think → act → observe, with every tool call the model asks for, not just the first.
- **3 tools:** live temperature (Open-Meteo API), Celsius → Fahrenheit, and a company policy search.
- **Arguments checked by Pydantic** before any tool runs; bad arguments become an error the model can read.
- **Failure recovery** — a failing tool becomes an `Error: ...` observation, never a crash; the policy search fails on its first call on purpose, so recovery happens on every run.
- **Hard step limit** — `MaxIterationsExceeded` is raised with the partial step log; the loop can never run forever.
- **Independent Verifier** — its own prompt, a structured `approved` / `reason` verdict, and the tools' evidence to check against.
- **One retry** — a rejected answer is retried once with the Verifier's reason; it's never shown as verified when it isn't.
- **Full step log** for every run, plus the Verifier's verdict, logged separately.

Demo

```
$ python main.py
ResearchHand — ask a question, or /quit to exit.
You: What's the weather in Lahore in Fahrenheit?
  step 0: get_weather({'city': 'Lahore'})
          -> Currently 18.0°C in Lahore
  step 1: celsius_to_fahrenheit({'celsius': 18.0})
          -> 64.4
Answer (verified): It's currently 64.4°F in Lahore.
You: What is your refund policy?
  step 0: search_docs({'query': 'refund policy'})
          -> Error: search index temporarily unavailable
  step 1: search_docs({'query': 'refund policy'})
          -> Refunds are accepted within 30 days of purchase, with the
             original receipt.
Answer (verified): Refunds are accepted within 30 days of purchase,
with the original receipt.
You: /quit
```

(Log lines are left out and long lines wrapped to keep the demo short. Model replies vary from run to run.)

Architecture



```
            ▼
   answer, labelled "verified" or "NOT verified"
```

- The Worker binds its tools to the model with LangChain's `bind_tools`, and runs the loop itself.
- The Verifier is an LCEL chain (`prompt | model.with_structured_output(VerifierVerdict)`).
- The Verifier sees only what the tools successfully returned — not the Worker's failed attempts — so it judges the answer, not the effort.
- `services/research_service.py` holds the Worker → Verifier handoff, separate from terminal input/output.

Project Structure

```
ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses/
├── data/
│   └── sample/
│       └── tasks.json          sample tasks for the script
├── scripts/
│   └── compare_with_tools.py   memory answer vs. verified answer
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── verifier.py     verify() -> VerifierVerdict
│   │   └── worker.py       run_worker() -> AgentResult
│   ├── chains/
│   │   ├── __init__.py
│   │   └── direct_answer.py build_direct_chain() — no tools
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py          get_model() -> ChatOpenAI
│   ├── prompts/
│   │   ├── direct.txt      the no-tools chain's prompt
│   │   ├── verifier.txt    the Verifier's prompt
│   │   └── worker.txt      the Worker's system prompt
│   ├── services/
│   │   ├── __init__.py
│   │   └── research_service.py answer_task(): Worker -> Verifier
│   ├── tools/
│   │   ├── __init__.py
│   │   ├── converter.py    celsius_to_fahrenheit (pure logic)
│   │   ├── docs_search.py  search_docs (fails on 1st call)
│   │   └── weather.py      get_weather (real API)
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── http_client.py  request_with_retry()
│   │   ├── logger.py       get_logger()
│   │   └── prompts.py      load_prompt(name)
│   ├── __init__.py
│   ├── config.py           Config, load_config()
│   ├── exceptions.py       all named errors
│   └── schemas.py          VerifierVerdict
├── tests/
│   ├── integration/
│   │   └── test_live_agents.py real API calls
│   └── unit/
│       ├── test_agents.py      no API key, no network
│       ├── test_config.py      no API key, no network
│       └── test_tools.py       no API key, no network
├── .env                    your real key — never committed
├── .env.example            the setting names, no real values
├── .gitignore              keeps .env, .venv/, __pycache__/ out of git
├── LICENSE                 lets others reuse your code (MIT)
├── main.py                 the terminal loop — the file you run
├── README.md               the repo's front page
└── requirements.txt        the 5 libraries, for pip install -r
```

Getting Started

Requirements: Python 3.10+ and an OpenAI API key. The weather API (Open-Meteo) is free and needs no key.

```
git clone <this-repo-url>
cd ResearchHand-Tool-Agent-Get-Verified-Answers-Not-Guesses
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env               # then put your key in .env
```

Configuration

Settings are read from `.env` (never committed):

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	`DEBUG`, `INFO`, `WARNING` or `ERROR`

The model (`gpt-4o-mini`) is set in one place: `MODEL_NAME` in `src/models/llm.py`. The step limit is `MAX_ITERATIONS` in `main.py`. Prompts are in `src/prompts/`. Weather works for the cities listed in `tools/weather.py`.

Usage

Run from the repo root:

```
python main.py
```

Ask about the weather (in °C or °F), the company's refund, shipping or warranty policy, or anything general. Type `/quit` to exit.

To compare a from-memory answer with the verified one for each sample task:

```
python -m scripts.compare_with_tools
```

Running the Tests

Run from the repo root:

```
# unit tests — no API key, no network
python -m tests.unit.test_config
python -m tests.unit.test_tools
python -m tests.unit.test_agents

# integration tests — real API calls, needs OPENAI_API_KEY
python -m tests.integration.test_live_agents
```

Every test case prints `OK` or `WRONG`. The unit tests check the config, the tools and the Worker's helpers. The integration tests check: the no-tools chain, the right tool for a clear task, no tool for general knowledge, recovery from a failing tool, the step limit, and that the Verifier **rejects** a deliberately wrong answer and approves a right one.

Error Handling

Situation	What happens
`OPENAI_API_KEY` missing	Exits at startup with one clear line
Invalid API key	Caught on the first call; exits cleanly
Rate limit	The OpenAI client retries with backoff; then the user is asked to wait
A tool fails or times out	Becomes an `Error: ...` observation; the Worker can retry or explain
The model sends bad tool arguments	Rejected by Pydantic before the tool runs; returned as an error
The model asks for an unknown tool	Returned as an error observation, not a crash

Situation	What happens
Task needs more than 5 rounds	`MaxIterationsExceeded` ; the partial step log is printed
Verifier rejects the answer	One retry with the reason; if still rejected, shown as NOT verified

Design Decisions

- **Hand-built loop instead of `create_agent`** — every model call, tool call and observation is visible and testable.
- **Tool errors as observations** — the model can react to a failure (retry, explain) instead of the program crashing.
- **A separate Verifier with its own prompt** — self-review shares the Worker's blind spots.
- **Evidence, not effort** — the Verifier gets the tools' successful results only, not the Worker's retries.
- **One retry, then label** — enough to fix a real slip, never an endless loop; a rejected answer is never shown as verified.
- **A deliberately flaky tool** — failure recovery is exercised on every run, not only during a real outage.

Limitations and Roadmap

- Weather is limited to a fixed list of cities (no geocoding).
- `search_docs` is a small local lookup, not real document retrieval.
- Memory lasts one question; there's no conversation history.
- Possible next steps: real document search (RAG), geocoding for any city, a LangGraph version of the loop, a web API.

License

MIT — see `LICENSE` .